

AVL tree

I. Core Property & Performance

- **Definition:** A self-balancing BST where the **Balance Factor** (BF) of every node is in $\{-1, 0, 1\}$.
- **Balance Factor:** $BF = \text{Height}(\text{Right Subtree}) - \text{Height}(\text{Left Subtree})$.
- **Performance:** All major operations (search, insert, delete) run in $O(\log N)$ time due to its height being $O(\log N)$.

II. Rebalancing Operations (Rotations)

Rotations are performed on the lowest unbalanced ancestor (Z) to restore $|BF| \leq 1$.

Type	Case	Imbalance Path	Rotation(s) on Z
Single	L-L (Left-Left)	Left child (Y), Left grandchild (X)	Right Rotation
Single	R-R (Right-Right)	Right child (Y), Right grandchild (X)	Left Rotation
Double	L-R (Left-Right)	Left child (Y), Right grandchild (X)	$\text{Left Rotation}(Y)$, then $\text{Right Rotation}(Z)$
Double	R-L (Right-Left)	Right child (Y), Left grandchild (X)	$\text{Right Rotation}(Y)$, then $\text{Left Rotation}(Z)$

III. STL Associative Containers

These C++ containers typically use **Red-Black Trees** (another $O(\log N)$ self-balancing BST).

Container	Allows Duplicates?	Stores Key-Value Pairs?
<code>std::set</code>	No	No (key only)
<code>std::multiset</code>	Yes	No (key only)
<code>std::map</code>	No	Yes (map)
<code>std::multimap</code>	Yes	Yes (map)